

EDA (Exploratory Data Analysis) Interview Questions & Answers Cheat Sheet

Table of Contents

1. [EDA Fundamentals](#)
2. [Data Understanding & Quality Assessment](#)
3. [Univariate Analysis](#)
4. [Bivariate Analysis](#)
5. [Multivariate Analysis](#)
6. [Data Visualization Techniques](#)
7. [Statistical Analysis in EDA](#)
8. [Outlier Detection & Treatment](#)
9. [Feature Engineering during EDA](#)
10. [Tools and Libraries](#)
11. [Business Context & Storytelling](#)
12. [Advanced EDA Techniques](#)

EDA Fundamentals

Q1: What is Exploratory Data Analysis and why is it crucial in data science?

Answer:

Definition:

Exploratory Data Analysis (EDA) is the critical process of investigating datasets to discover patterns, spot anomalies, test hypotheses, and summarize main characteristics using statistical and visual methods.

Key Objectives:

1. **Data Understanding:** Get familiar with data structure, types, and quality
2. **Pattern Discovery:** Identify trends, relationships, and hidden insights
3. **Anomaly Detection:** Find outliers, missing values, and inconsistencies
4. **Hypothesis Formation:** Generate questions and assumptions for further analysis
5. **Model Preparation:** Clean and prepare data for machine learning

EDA Process Framework:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

class EDAFramework:
    def __init__(self, df):
        self.df = df
        self.report = {}

    def data_overview(self):
        """Get basic information about the dataset"""
        overview = {
            'shape': self.df.shape,
            'columns': list(self.df.columns),
            'dtypes': self.df.dtypes.to_dict(),
            'memory_usage': self.df.memory_usage(deep=True).sum(),
            'null_counts': self.df.isnull().sum().to_dict(),
            'duplicate_rows': self.df.duplicated().sum()
        }

        print("=== DATASET OVERVIEW ===")
        print(f"Shape: {overview['shape']}")
        print(f"Memory Usage: {overview['memory_usage'] / 1024**2:.2f} MB")
        print(f"Duplicate Rows: {overview['duplicate_rows']}")
        print(f"Columns with Missing Values: {sum(1 for v in overview['null_counts'].values if v > 0)}")

        return overview

    def data_quality_assessment(self):
        """Assess data quality issues"""
        quality_report = {}

        for col in self.df.columns:
            col_info = {
                'missing_count': self.df[col].isnull().sum(),
                'missing_percentage': (self.df[col].isnull().sum() / len(self.df)) * 100,
                'unique_count': self.df[col].nunique(),
                'data_type': str(self.df[col].dtype)
            }

            # Additional checks for numeric columns
            if pd.api.types.is_numeric_dtype(self.df[col]):
                col_info.update({
                    'zero_count': (self.df[col] == 0).sum(),
                    'negative_count': (self.df[col] < 0).sum(),
                    'infinite_count': np.isinf(self.df[col]).sum()
                })

            # Additional checks for string columns
            elif pd.api.types.is_object_dtype(self.df[col]):
                col_info.update({
                    'empty_strings': (self.df[col] == '').sum(),
                    'whitespace_only': self.df[col].str.isspace().sum() if self.df[col].dtype == 'object' else 0
                })

            quality_report[col] = col_info

        return quality_report

```

```

        quality_report[col] = col_info

    # Create quality summary
    high_missing = [col for col, info in quality_report.items()
                     if info['missing_percentage'] > 20]

    print("=== DATA QUALITY ASSESSMENT ===")
    if high_missing:
        print(f"Columns with >20% missing values: {high_missing}")

    return quality_report

def statistical_summary(self):
    """Generate statistical summary for all variables"""

    # Numeric variables
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    if len(numeric_cols) > 0:
        print("=== NUMERIC VARIABLES SUMMARY ===")
        numeric_summary = self.df[numeric_cols].describe()
        print(numeric_summary)

    # Categorical variables
    categorical_cols = self.df.select_dtypes(include=['object', 'category']).columns
    if len(categorical_cols) > 0:
        print("\n=== CATEGORICAL VARIABLES SUMMARY ===")
        for col in categorical_cols:
            print(f"\n{col}:")
            print(f"    Unique values: {self.df[col].nunique()}")
            print(f"    Most frequent: {self.df[col].mode().iloc[0]} if len(self.df[col].mode()) > 1")
            print(f"    Top 5 values:")
            print(self.df[col].value_counts().head())

    return {
        'numeric_summary': numeric_summary if len(numeric_cols) > 0 else None,
        'categorical_summary': {col: self.df[col].value_counts() for col in categorical_cols}
    }

# Example usage
def demonstrate_eda_framework():
    # Create sample dataset
    np.random.seed(42)
    n_samples = 1000

    data = {
        'age': np.random.normal(35, 12, n_samples).clip(18, 80),
        'income': np.random.lognormal(10.5, 0.8, n_samples),
        'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples, p=[0.4, 0.3, 0.2, 0.1]),
        'satisfaction': np.random.normal(7, 2, n_samples).clip(1, 10),
        'city': np.random.choice(['NYC', 'LA', 'Chicago', 'Houston', 'Phoenix'], n_samples)
    }

    df = pd.DataFrame(data)

    # Introduce some missing values and outliers

```

```

df.loc[np.random.choice(df.index, 50, replace=False), 'income'] = np.nan
df.loc[np.random.choice(df.index, 30, replace=False), 'satisfaction'] = np.nan

# Run EDA framework
eda = EDAFramework(df)
overview = eda.data_overview()
quality = eda.data_quality_assessment()
summary = eda.statistical_summary()

return eda, df

# Run demonstration
eda_demo, sample_df = demonstrate_eda_framework()

```

Q2: What are the different types of EDA and when do you use each?

Answer:

Types of EDA:

Type	Focus	Techniques	When to Use
Univariate	Single variable	Histograms, box plots, summary stats	Initial data understanding
Bivariate	Two variables	Scatter plots, correlation, cross-tabs	Relationship exploration
Multivariate	Multiple variables	Pair plots, PCA, clustering	Complex pattern detection

Detailed Implementation:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency, pearsonr, spearmanr
import warnings
warnings.filterwarnings('ignore')

class EDATypes:
    def __init__(self, df):
        self.df = df

    def univariate_analysis(self, column=None):
        """Comprehensive univariate analysis"""

        if column is None:
            # Analyze all columns
            columns_to_analyze = self.df.columns
        else:
            columns_to_analyze = [column]

        results = {}

        for col in columns_to_analyze:
            col_result = {}

```

```

        if pd.api.types.is_numeric_dtype(self.df[col]):
            # Numeric variable analysis
            col_result = self._analyze_numeric_univariate(col)
        else:
            # Categorical variable analysis
            col_result = self._analyze_categorical_univariate(col)

        results[col] = col_result

    return results

def _analyze_numeric_univariate(self, col):
    """Analyze numeric variable"""
    data = self.df[col].dropna()

    # Descriptive statistics
    stats = {
        'count': len(data),
        'mean': data.mean(),
        'median': data.median(),
        'mode': data.mode().iloc[0] if len(data.mode()) > 0 else np.nan,
        'std': data.std(),
        'var': data.var(),
        'min': data.min(),
        'max': data.max(),
        'range': data.max() - data.min(),
        'q25': data.quantile(0.25),
        'q75': data.quantile(0.75),
        'iqr': data.quantile(0.75) - data.quantile(0.25),
        'skewness': data.skew(),
        'kurtosis': data.kurtosis()
    }

    # Distribution shape analysis
    if abs(stats['skewness']) < 0.5:
        distribution_shape = "Approximately symmetric"
    elif stats['skewness'] > 0.5:
        distribution_shape = "Right-skewed (positively skewed)"
    else:
        distribution_shape = "Left-skewed (negatively skewed)"

    stats['distribution_shape'] = distribution_shape

    # Outlier detection using IQR method
    q1, q3 = stats['q25'], stats['q75']
    iqr = stats['iqr']
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    outliers = data[(data < lower_bound) | (data > upper_bound)]
    stats['outlier_count'] = len(outliers)
    stats['outlier_percentage'] = (len(outliers) / len(data)) * 100

    return stats

```

```

def _analyze_categorical_univariate(self, col):
    """Analyze categorical variable"""
    data = self.df[col].dropna()
    value_counts = data.value_counts()

    stats = {
        'count': len(data),
        'unique_values': data.nunique(),
        'most_frequent': value_counts.index[0],
        'most_frequent_count': value_counts.iloc[0],
        'least_frequent': value_counts.index[-1],
        'least_frequent_count': value_counts.iloc[-1],
        'mode_percentage': (value_counts.iloc[0] / len(data)) * 100
    }

    # Check for high cardinality
    if stats['unique_values'] > len(data) * 0.7:
        stats['cardinality'] = "High (potentially ID-like)"
    elif stats['unique_values'] < 10:
        stats['cardinality'] = "Low (good for categorization)"
    else:
        stats['cardinality'] = "Medium"

    stats['value_distribution'] = value_counts.to_dict()

    return stats

def bivariate_analysis(self, var1, var2):
    """Comprehensive bivariate analysis"""

    # Clean data
    clean_data = self.df[[var1, var2]].dropna()

    result = {
        'variable_1': var1,
        'variable_2': var2,
        'sample_size': len(clean_data)
    }

    # Determine variable types
    var1_numeric = pd.api.types.is_numeric_dtype(clean_data[var1])
    var2_numeric = pd.api.types.is_numeric_dtype(clean_data[var2])

    if var1_numeric and var2_numeric:
        # Numeric vs Numeric
        result.update(self._numeric_vs_numeric(clean_data, var1, var2))
        result['analysis_type'] = 'Numeric vs Numeric'

    elif var1_numeric and not var2_numeric:
        # Numeric vs Categorical
        result.update(self._numeric_vs_categorical(clean_data, var1, var2))
        result['analysis_type'] = 'Numeric vs Categorical'

    elif not var1_numeric and var2_numeric:
        # Categorical vs Numeric
        result.update(self._numeric_vs_categorical(clean_data, var2, var1))

```

```

        result['analysis_type'] = 'Categorical vs Numeric'

    else:
        # Categorical vs Categorical
        result.update(self._categorical_vs_categorical(clean_data, var1, var2))
        result['analysis_type'] = 'Categorical vs Categorical'

    return result

def _numeric_vs_numeric(self, data, var1, var2):
    """Analyze two numeric variables"""

    # Correlation analysis
    pearson_corr, pearson_p = pearsonr(data[var1], data[var2])
    spearman_corr, spearman_p = spearmanr(data[var1], data[var2])

    # Determine correlation strength
    abs_pearson = abs(pearson_corr)
    if abs_pearson < 0.3:
        strength = "Weak"
    elif abs_pearson < 0.7:
        strength = "Moderate"
    else:
        strength = "Strong"

    return {
        'pearson_correlation': pearson_corr,
        'pearson_p_value': pearson_p,
        'spearman_correlation': spearman_corr,
        'spearman_p_value': spearman_p,
        'correlation_strength': strength,
        'relationship_type': 'Linear' if abs(pearson_corr) > abs(spearman_corr) el
    }

def _numeric_vs_categorical(self, data, numeric_var, categorical_var):
    """Analyze numeric vs categorical variables"""

    # Group statistics
    group_stats = data.groupby(categorical_var)[numeric_var].agg([
        'count', 'mean', 'median', 'std', 'min', 'max'
    ]).round(3)

    # ANOVA test (if more than 2 groups)
    groups = [group[numeric_var].values for name, group in data.groupby(categorical_v

    if len(groups) > 2:
        from scipy.stats import f_oneway
        f_stat, p_value = f_oneway(*groups)
        test_name = "ANOVA"
    else:
        from scipy.stats import ttest_ind
        f_stat, p_value = ttest_ind(groups[0], groups[1])
        test_name = "t-test"

    return {
        'group_statistics': group_stats.to_dict(),

```

```

        'statistical_test': test_name,
        'test_statistic': f_stat,
        'p_value': p_value,
        'significant_difference': p_value < 0.05
    }

def _categorical_vs_categorical(self, data, var1, var2):
    """Analyze two categorical variables"""

    # Cross-tabulation
    crosstab = pd.crosstab(data[var1], data[var2])

    # Chi-square test
    chi2, p_value, dof, expected = chi2_contingency(crosstab)

    # Cramér's V (effect size)
    n = crosstab.sum().sum()
    cramers_v = np.sqrt(chi2 / (n * (min(crosstab.shape) - 1)))

    return {
        'crosstab': crosstab.to_dict(),
        'chi_square_statistic': chi2,
        'p_value': p_value,
        'degrees_of_freedom': dof,
        'cramers_v': cramers_v,
        'association_strength': self._interpret_cramers_v(cramers_v),
        'significant_association': p_value < 0.05
    }

def _interpret_cramers_v(self, cramers_v):
    """Interpret Cramér's V effect size"""
    if cramers_v < 0.1:
        return "Negligible"
    elif cramers_v < 0.3:
        return "Small"
    elif cramers_v < 0.5:
        return "Medium"
    else:
        return "Large"

def multivariate_analysis(self, variables=None):
    """Multivariate analysis techniques"""

    if variables is None:
        # Use all numeric variables
        variables = self.df.select_dtypes(include=[np.number]).columns.tolist()

    if len(variables) < 3:
        return "Multivariate analysis requires at least 3 variables"

    # Correlation matrix
    correlation_matrix = self.df[variables].corr()

    # Principal Component Analysis
    from sklearn.decomposition import PCA
    from sklearn.preprocessing import StandardScaler

```

```

    # Standardize the data
    scaler = StandardScaler()
    scaled_data = scaler.fit_transform(self.df[variables].dropna())

    # PCA
    pca = PCA()
    pca_result = pca.fit_transform(scaled_data)

    # Explained variance
    explained_variance = pca.explained_variance_ratio_
    cumulative_variance = np.cumsum(explained_variance)

    # Find number of components for 95% variance
    n_components_95 = np.argmax(cumulative_variance >= 0.95) + 1

    return {
        'correlation_matrix': correlation_matrix,
        'pca_explained_variance': explained_variance,
        'pca_cumulative_variance': cumulative_variance,
        'components_for_95_variance': n_components_95,
        'original_dimensions': len(variables),
        'recommended_dimensions': n_components_95
    }

# Example usage and visualization
def demonstrate_eda_types():
    # Load sample data
    tips = sns.load_dataset('tips')

    eda_analyzer = EDATypes(tips)

    # Univariate analysis
    print("=== UNIVARIATE ANALYSIS ===")
    univariate_results = eda_analyzer.univariate_analysis('total_bill')
    for key, value in univariate_results['total_bill'].items():
        print(f"{key}: {value}")

    print("\n=== BIVARIATE ANALYSIS ===")
    # Bivariate analysis
    bivariate_results = eda_analyzer.bivariate_analysis('total_bill', 'tip')
    for key, value in bivariate_results.items():
        if key not in ['variable_1', 'variable_2']:
            print(f"{key}: {value}")

    print("\n=== MULTIVARIATE ANALYSIS ===")
    # Multivariate analysis
    multivariate_results = eda_analyzer.multivariate_analysis(['total_bill', 'tip', 'size'])
    print(f"Original dimensions: {multivariate_results['original_dimensions']}")
    print(f"Components needed for 95% variance: {multivariate_results['components_for_95_"]

    return eda_analyzer, tips

# Run demonstration
analyzer, data = demonstrate_eda_types()

```

Data Understanding & Quality Assessment

Q3: How do you assess data quality and what are the common data quality issues?

Answer:

Data Quality Dimensions:

Dimension	Description	Assessment Methods
Completeness	Missing data	Null counts, missing patterns
Accuracy	Correctness	Domain validation, outlier detection
Consistency	Uniformity	Format checking, duplicate detection
Timeliness	Currency	Date validation, temporal analysis
Validity	Constraint compliance	Data type, range validation
Uniqueness	No duplicates	Duplicate identification

Comprehensive Data Quality Assessment:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import re

class DataQualityAssessment:
    def __init__(self, df):
        self.df = df
        self.quality_report = {}

    def assess_completeness(self):
        """Assess missing data patterns"""

        missing_summary = {}

        for col in self.df.columns:
            missing_count = self.df[col].isnull().sum()
            missing_pct = (missing_count / len(self.df)) * 100

            missing_summary[col] = {
                'missing_count': missing_count,
                'missing_percentage': missing_pct,
                'complete_count': len(self.df) - missing_count
            }

        # Missing data patterns
        missing_patterns = self.df.isnull().value_counts().head(10)

        # Columns with high missing rates
```

```

high_missing = {k: v for k, v in missing_summary.items()
                 if v['missing_percentage'] > 20}

print("=== COMPLETENESS ASSESSMENT ===")
print(f"Columns with >20% missing data: {len(high_missing)}")

if high_missing:
    for col, info in high_missing.items():
        print(f"  {col}: {info['missing_percentage']:.1f}% missing")

# Visualize missing data
self._visualize_missing_data()

return {
    'missing_summary': missing_summary,
    'high_missing_columns': high_missing,
    'missing_patterns': missing_patterns
}

def assess_accuracy(self):
    """Assess data accuracy through outlier detection and domain validation"""

    accuracy_issues = {}

    for col in self.df.columns:
        col_issues = []

        if pd.api.types.is_numeric_dtype(self.df[col]):
            # Outlier detection for numeric columns
            outliers = self._detect_outliers_iqr(self.df[col])
            if len(outliers) > 0:
                col_issues.append(f"{len(outliers)} potential outliers detected")

            # Check for impossible values (domain-specific)
            if col.lower() in ['age'] and self.df[col].max() > 120:
                col_issues.append("Age values > 120 detected")

            if col.lower() in ['salary', 'income', 'price'] and (self.df[col] < 0).any():
                col_issues.append("Negative values in positive-only field")

        elif pd.api.types.is_object_dtype(self.df[col]):
            # Check for inconsistent formats
            if col.lower() in ['email'] and not self._validate_email_format(self.df[col]):
                col_issues.append("Invalid email formats detected")

            if col.lower() in ['phone', 'mobile'] and not self._validate_phone_format(self.df[col]):
                col_issues.append("Invalid phone formats detected")

        if col_issues:
            accuracy_issues[col] = col_issues

    return accuracy_issues

def assess_consistency(self):
    """Assess data consistency"""

```

```

consistency_issues = {}

for col in self.df.columns:
    issues = []

    if pd.api.types.is_object_dtype(self.df[col]):
        # Check for inconsistent casing
        values = self.df[col].dropna().astype(str)
        unique_lower = values.str.lower().nunique()
        unique_original = values.nunique()

        if unique_lower < unique_original:
            issues.append(f"Inconsistent casing: {unique_original - unique_lower}")

        # Check for leading/trailing spaces
        has_spaces = values.str.strip().nunique() < values.nunique()
        if has_spaces:
            issues.append("Leading/trailing spaces detected")

        # Check for multiple formats in same column
        if self._detect_mixed_formats(values):
            issues.append("Mixed data formats detected")

    if issues:
        consistency_issues[col] = issues

return consistency_issues

def assess_uniqueness(self):
    """Assess uniqueness constraints"""

    # Overall duplicate rows
    duplicate_rows = self.df.duplicated().sum()

    # Column-level uniqueness
    uniqueness_report = {}

    for col in self.df.columns:
        total_count = len(self.df[col].dropna())
        unique_count = self.df[col].nunique()
        duplicate_count = total_count - unique_count

        uniqueness_report[col] = {
            'total_values': total_count,
            'unique_values': unique_count,
            'duplicate_values': duplicate_count,
            'uniqueness_ratio': unique_count / total_count if total_count > 0 else 0
        }

    # Identify potential ID columns (high uniqueness)
    potential_ids = {k: v for k, v in uniqueness_report.items()
                     if v['uniqueness_ratio'] > 0.95}

    return {
        'duplicate_rows': duplicate_rows,
        'column_uniqueness': uniqueness_report,
        'potential_ids': potential_ids
    }

```

```

        'potential_id_columns': potential_ids
    }

def assess_validity(self):
    """Assess data validity (domain constraints)"""

    validity_issues = {}

    for col in self.df.columns:
        issues = []

        # Data type consistency
        if pd.api.types.is_numeric_dtype(self.df[col]):
            # Check for infinite values
            inf_count = np.isinf(self.df[col]).sum()
            if inf_count > 0:
                issues.append(f"{inf_count} infinite values")

            # Check for very large values (potential data entry errors)
            if self.df[col].max() > 1e10:
                issues.append("Extremely large values detected")

        # Date validation
        if col.lower() in ['date', 'timestamp'] or 'date' in col.lower():
            date_issues = self._validate_dates(self.df[col])
            if date_issues:
                issues.extend(date_issues)

        if issues:
            validity_issues[col] = issues

    return validity_issues

def generate_quality_report(self):
    """Generate comprehensive data quality report"""

    report = {
        'completeness': self.assess_completeness(),
        'accuracy': self.assess_accuracy(),
        'consistency': self.assess_consistency(),
        'uniqueness': self.assess_uniqueness(),
        'validity': self.assess_validity()
    }

    # Overall quality score
    total_issues = (
        len(report['completeness']['high_missing_columns']) +
        len(report['accuracy']) +
        len(report['consistency']) +
        (1 if report['uniqueness']['duplicate_rows'] > 0 else 0) +
        len(report['validity'])
    )

    total_columns = len(self.df.columns)
    quality_score = max(0, 100 - (total_issues / total_columns) * 10)

```

```

report['overall_quality_score'] = quality_score

print("=== DATA QUALITY REPORT ===")
print(f"Overall Quality Score: {quality_score:.1f}/100")
print(f"Total Issues Found: {total_issues}")

return report

def _detect_outliers_iqr(self, series, multiplier=1.5):
    """Detect outliers using IQR method"""
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - multiplier * IQR
    upper_bound = Q3 + multiplier * IQR

    outliers = series[(series < lower_bound) | (series > upper_bound)]
    return outliers.index.tolist()

def _validate_email_format(self, series):
    """Validate email format"""
    email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    valid_emails = series.dropna().astype(str).str.match(email_pattern)
    return valid_emails.all()

def _validate_phone_format(self, series):
    """Validate phone format (basic)"""
    phone_pattern = r'^\+?[\d\s\-\(\)]{10,}$'
    valid_phones = series.dropna().astype(str).str.match(phone_pattern)
    return valid_phones.all()

def _detect_mixed_formats(self, series):
    """Detect mixed formats in string column"""
    # Simple check for mixed numeric/text
    sample = series.head(100)
    has_numbers = sample.str.contains(r'\d', na=False).any()
    has_letters = sample.str.contains(r'[a-zA-Z]', na=False).any()

    return has_numbers and has_letters

def _validate_dates(self, series):
    """Validate date values"""
    issues = []

    if pd.api.types.is_datetime64_any_dtype(series):
        # Check for future dates (if applicable)
        future_dates = series > datetime.now()
        if future_dates.any():
            issues.append(f"{future_dates.sum()} future dates detected")

        # Check for very old dates
        very_old = series < datetime(1900, 1, 1)
        if very_old.any():
            issues.append(f"{very_old.sum()} dates before 1900 detected")

```

```

        return issues

def _visualize_missing_data(self):
    """Visualize missing data patterns"""

    missing_df = self.df.isnull()

    if missing_df.any().any(): # If there are any missing values
        fig, axes = plt.subplots(1, 2, figsize=(15, 6))

        # Missing data heatmap
        sns.heatmap(missing_df, cbar=True, ax=axes[0], cmap='viridis')
        axes[0].set_title('Missing Data Pattern')
        axes[0].set_xlabel('Columns')
        axes[0].set_ylabel('Rows')

        # Missing data bar chart
        missing_counts = self.df.isnull().sum()
        missing_counts = missing_counts[missing_counts > 0].sort_values(ascending=

        if len(missing_counts) > 0:
            missing_counts.plot(kind='bar', ax=axes[1])
            axes[1].set_title('Missing Data Count by Column')
            axes[1].set_xlabel('Columns')
            axes[1].set_ylabel('Missing Count')
            axes[1].tick_params(axis='x', rotation=45)

        plt.tight_layout()
        plt.show()

# Example usage
def demonstrate_data_quality_assessment():
    # Create sample dataset with quality issues
    np.random.seed(42)
    n_samples = 1000

    # Create data with intentional quality issues
    data = {
        'id': range(1, n_samples + 1),
        'name': ['User_' + str(i) for i in range(n_samples)],
        'age': np.random.normal(35, 15, n_samples),
        'email': ['user' + str(i) + '@email.com' for i in range(n_samples)],
        'salary': np.random.normal(50000, 15000, n_samples),
        'join_date': pd.date_range('2020-01-01', periods=n_samples, freq='D')
    }

    df = pd.DataFrame(data)

    # Introduce quality issues
    # Missing values
    df.loc[np.random.choice(df.index, 50), 'email'] = np.nan
    df.loc[np.random.choice(df.index, 30), 'salary'] = np.nan

    # Outliers
    df.loc[np.random.choice(df.index, 10), 'age'] = np.random.uniform(150, 200, 10)
    df.loc[np.random.choice(df.index, 5), 'salary'] = -1000 # Negative salary

```

```

# Inconsistent formatting
df.loc[np.random.choice(df.index, 20), 'name'] = df.loc[np.random.choice(df.index, 20), 'name'].str.upper

# Duplicates
df = pd.concat([df, df.iloc[0:5]], ignore_index=True)

# Assess data quality
dqa = DataQualityAssessment(df)
quality_report = dqa.generate_quality_report()

return dqa, df, quality_report

# Run demonstration
assessor, sample_data, quality_rep = demonstrate_data_quality_assessment()

```

Q4: How do you handle missing data during EDA?

Answer:

Missing Data Types:

Type	Description	Example	Handling Strategy
MCAR	Missing Completely at Random	Random sensor failures	Simple imputation acceptable
MAR	Missing at Random	Income missing for unemployed	Conditional imputation
MNAR	Missing Not at Random	High earners hide income	Domain knowledge needed

Comprehensive Missing Data Analysis:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from scipy.stats import chi2_contingency
import missingno as msno

class MissingDataHandler:
    def __init__(self, df):
        self.df = df
        self.missing_analysis = {}

    def analyze_missing_patterns(self):
        """Comprehensive missing data pattern analysis"""

        # Basic missing data statistics
        missing_stats = {}
        for col in self.df.columns:

```

```

        missing_count = self.df[col].isnull().sum()
        missing_pct = (missing_count / len(self.df)) * 100

        missing_stats[col] = {
            'count': missing_count,
            'percentage': missing_pct,
            'data_type': str(self.df[col].dtype)
        }

    # Missing data patterns
    missing_patterns = self.df.isnull().value_counts()

    # Correlation between missing values
    missing_corr = self.df.isnull().corr()

    print("=== MISSING DATA ANALYSIS ===")
    print(f"Total rows: {len(self.df)}")
    print(f"Columns with missing data: {sum(1 for stats in missing_stats.values() if

    # Show columns with missing data
    missing_cols = {k: v for k, v in missing_stats.items() if v['count'] > 0}
    if missing_cols:
        print("\nColumns with missing values:")
        for col, stats in missing_cols.items():
            print(f"  {col}: {stats['count']} ({stats['percentage']:.1f}%)")

    return {
        'missing_statistics': missing_stats,
        'missing_patterns': missing_patterns,
        'missing_correlation': missing_corr
    }

def test_missing_mechanism(self, target_col, test_cols=None):
    """Test if missing data is MAR or MCAR"""

    if test_cols is None:
        test_cols = [col for col in self.df.columns if col != target_col]

    results = {}

    # Create missing indicator
    missing_indicator = self.df[target_col].isnull()

    for col in test_cols:
        if pd.api.types.is_numeric_dtype(self.df[col]):
            # For numeric variables, use t-test
            from scipy.stats import ttest_ind

            observed = self.df[col][~missing_indicator].dropna()
            missing = self.df[col][missing_indicator].dropna()

            if len(observed) > 0 and len(missing) > 0:
                stat, p_value = ttest_ind(observed, missing)
                results[col] = {
                    'test': 't-test',
                    'statistic': stat,

```

```

        'p_value': p_value,
        'significant': p_value < 0.05
    }
else:
    # For categorical variables, use chi-square test
    crosstab = pd.crosstab(self.df[col], missing_indicator)
    chi2, p_value, dof, expected = chi2_contingency(crosstab)

    results[col] = {
        'test': 'chi-square',
        'statistic': chi2,
        'p_value': p_value,
        'significant': p_value < 0.05
    }

# Interpretation
significant_tests = sum(1 for result in results.values() if result['significant'])

if significant_tests == 0:
    mechanism = "Likely MCAR (Missing Completely at Random)"
else:
    mechanism = f"Likely MAR (Missing at Random) - {significant_tests} significant tests"

print(f"\n=== MISSING MECHANISM TEST for {target_col} ===")
print(f"Result: {mechanism}")

return {
    'mechanism': mechanism,
    'test_results': results
}

def visualize_missing_data(self):
    """Create comprehensive visualizations for missing data"""

    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # Missing data matrix
    msno.matrix(self.df, ax=axes[0,0])
    axes[0,0].set_title('Missing Data Matrix')

    # Missing data heatmap
    missing_corr = self.df.isnull().corr()
    sns.heatmap(missing_corr, annot=True, cmap='coolwarm', center=0, ax=axes[0,1])
    axes[0,1].set_title('Missing Data Correlation')

    # Missing data bar chart
    missing_counts = self.df.isnull().sum()
    missing_counts = missing_counts[missing_counts > 0].sort_values(ascending=True)

    if len(missing_counts) > 0:
        missing_counts.plot(kind='barh', ax=axes[1,0])
        axes[1,0].set_title('Missing Data Count by Column')
        axes[1,0].set_xlabel('Missing Count')

    # Missing data percentage
    missing_pct = (self.df.isnull().sum() / len(self.df)) * 100

```

```

missing_pct = missing_pct[missing_pct > 0].sort_values(ascending=True)

if len(missing_pct) > 0:
    missing_pct.plot(kind='barh', ax=axes[1,1])
    axes[1,1].set_title('Missing Data Percentage by Column')
    axes[1,1].set_xlabel('Missing Percentage (%)')

plt.tight_layout()
plt.show()

def imputation_strategies(self, column, method='simple'):
    """Apply different imputation strategies"""

    if column not in self.df.columns:
        return "Column not found"

    original_missing = self.df[column].isnull().sum()
    strategies_results = {}

    # Strategy 1: Simple Imputation
    if method in ['simple', 'all']:
        if pd.api.types.is_numeric_dtype(self.df[column]):
            # For numeric: mean, median, mode
            simple_strategies = {
                'mean': self.df[column].fillna(self.df[column].mean()),
                'median': self.df[column].fillna(self.df[column].median()),
                'mode': self.df[column].fillna(self.df[column].mode().iloc[0] if len(
            }
        else:
            # For categorical: mode, forward fill
            simple_strategies = {
                'mode': self.df[column].fillna(self.df[column].mode().iloc[0] if len(
                'forward_fill': self.df[column].fillna(method='ffill'),
                'constant': self.df[column].fillna('Unknown')
            }

        strategies_results['simple'] = simple_strategies

    # Strategy 2: KNN Imputation (for numeric data)
    if method in ['knn', 'all'] and pd.api.types.is_numeric_dtype(self.df[column]):
        numeric_cols = self.df.select_dtypes(include=[np.number]).columns
        knn_imputer = KNNImputer(n_neighbors=5)

        imputed_data = knn_imputer.fit_transform(self.df[numeric_cols])
        col_idx = list(numeric_cols).index(column)

        strategies_results['knn'] = pd.Series(imputed_data[:, col_idx], index=self.df

    # Strategy 3: Iterative Imputation (MICE)
    if method in ['iterative', 'all'] and pd.api.types.is_numeric_dtype(self.df[column]):
        numeric_cols = self.df.select_dtypes(include=[np.number]).columns
        iterative_imputer = IterativeImputer(random_state=42)

        imputed_data = iterative_imputer.fit_transform(self.df[numeric_cols])
        col_idx = list(numeric_cols).index(column)

```

```

        strategies_results['iterative'] = pd.Series(imputed_data[:, col_idx], index=s

# Compare strategies
comparison_results = {}
for strategy_name, strategy_data in strategies_results.items():
    if isinstance(strategy_data, dict):
        # Simple strategies
        for substrategy_name, imputed_series in strategy_data.items():
            comparison_results[f"{strategy_name}_{substrategy_name}"] = {
                'missing_after': imputed_series.isnull().sum(),
                'mean': imputed_series.mean() if pd.api.types.is_numeric_dtype(im
                'std': imputed_series.std() if pd.api.types.is_numeric_dtype(impu
            }
    else:
        # Advanced strategies
        comparison_results[strategy_name] = {
            'missing_after': strategy_data.isnull().sum(),
            'mean': strategy_data.mean() if pd.api.types.is_numeric_dtype(strateg
            'std': strategy_data.std() if pd.api.types.is_numeric_dtype(strategy_

}

print(f"\n=== IMPUTATION COMPARISON for {column} ===")
print(f"Original missing values: {original_missing}")

for strategy, stats in comparison_results.items():
    print(f"\n{strategy}:")
    print(f"Missing after imputation: {stats['missing_after']}")
    if stats['mean'] is not None:
        print(f"Mean: {stats['mean']:.3f}")
        print(f"Std: {stats['std']:.3f}")

return {
    'strategies': strategies_results,
    'comparison': comparison_results
}

def recommend_strategy(self, column):
    """Recommend imputation strategy based on data characteristics"""

    if column not in self.df.columns:
        return "Column not found"

    missing_pct = (self.df[column].isnull().sum() / len(self.df)) * 100
    data_type = str(self.df[column].dtype)

    recommendations = []

    # Based on missing percentage
    if missing_pct < 5:
        recommendations.append("Low missing rate - simple imputation or deletion acco
    elif missing_pct < 20:
        recommendations.append("Moderate missing rate - use advanced imputation techn
    else:
        recommendations.append("High missing rate - consider dropping column or using

    # Based on data type

```

```

        if pd.api.types.is_numeric_dtype(self.df[column]):
            if self.df[column].skew() > 1:
                recommendations.append("Highly skewed - use median imputation")
            else:
                recommendations.append("Normally distributed - mean imputation acceptable")
        else:
            recommendations.append("Categorical - use mode or create 'Unknown' category")

    # Based on correlation with other variables
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    if column in numeric_cols and len(numeric_cols) > 1:
        corr_with_others = abs(self.df[numeric_cols].corr()[column]).drop(column).max()
        if corr_with_others > 0.5:
            recommendations.append("High correlation with other variables - use KNN")

    print(f"\n=== RECOMMENDATIONS for {column} ===")
    print(f"Missing percentage: {missing_pct:.1f}%")
    print(f>Data type: {data_type}")
    print("\nRecommendations:")
    for i, rec in enumerate(recommendations, 1):
        print(f"  {i}. {rec}")

    return recommendations

# Example usage
def demonstrate_missing_data_handling():
    # Create dataset with different types of missing data
    np.random.seed(42)
    n_samples = 1000

    # Base data
    age = np.random.normal(35, 10, n_samples)
    income = np.random.normal(50000, 15000, n_samples)
    education = np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n_samples)

    df = pd.DataFrame({
        'age': age,
        'income': income,
        'education': education,
        'satisfaction': np.random.normal(7, 2, n_samples)
    })

    # Introduce missing data with different mechanisms

    # MCAR: Random missing in satisfaction
    mcar_indices = np.random.choice(df.index, size=int(0.1 * n_samples), replace=False)
    df.loc[mcar_indices, 'satisfaction'] = np.nan

    # MAR: Income missing for people with high school education
    mar_condition = df['education'] == 'High School'
    mar_indices = df[mar_condition].index
    mar_sample = np.random.choice(mar_indices, size=int(0.3 * len(mar_indices)), replace=False)
    df.loc[mar_sample, 'income'] = np.nan

    # MNAR: High income individuals don't report income
    mnar_condition = df['income'] > df['income'].quantile(0.8)

```

```

mnar_indices = df[mnar_condition].index
mnar_sample = np.random.choice(mnar_indices, size=int(0.4 * len(mnar_indices)), replace=True)
df.loc[mnar_sample, 'income'] = np.nan

# Analyze missing data
handler = MissingDataHandler(df)

# Basic analysis
missing_analysis = handler.analyze_missing_patterns()

# Test missing mechanism
income_mechanism = handler.test_missing_mechanism('income')

# Visualize
handler.visualize_missing_data()

# Compare imputation strategies
income_imputation = handler.imputation_strategies('income', 'all')

# Get recommendations
income_recommendations = handler.recommend_strategy('income')

return handler, df

# Run demonstration
missing_handler, missing_data = demonstrate_missing_data_handling()

```

Univariate Analysis

Q5: How do you perform comprehensive univariate analysis for different data types?

Answer:

Univariate Analysis Framework:

Data Type	Key Metrics	Visualizations	Statistical Tests
Numeric	Central tendency, spread, shape	Histogram, box plot, QQ plot	Normality tests
Categorical	Frequency, proportion	Bar chart, pie chart	Chi-square goodness of fit
Ordinal	Median, percentiles	Bar chart, cumulative plot	Mann-Whitney U test
Datetime	Trends, seasonality	Time series plot, seasonal decomposition	Trend tests

Comprehensive Univariate Analysis Implementation:

```

import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import normaltest, shapiro, jarque_bera, anderson
import warnings
warnings.filterwarnings('ignore')

class UnivariateAnalysis:
    def __init__(self, df):
        self.df = df

    def analyze_numeric_variable(self, column, create_plots=True):
        """Comprehensive analysis of numeric variables"""

        if column not in self.df.columns:
            return f"Column '{column}' not found"

        if not pd.api.types.is_numeric_dtype(self.df[column]):
            return f"Column '{column}' is not numeric"

        data = self.df[column].dropna()

        if len(data) == 0:
            return f"No non-null values in column '{column}'"

        # Descriptive Statistics
        stats_summary = {
            'count': len(data),
            'missing': self.df[column].isnull().sum(),
            'missing_pct': (self.df[column].isnull().sum() / len(self.df)) * 100,

            # Central Tendency
            'mean': data.mean(),
            'median': data.median(),
            'mode': data.mode().iloc[0] if len(data.mode()) > 0 else np.nan,

            # Dispersion
            'std': data.std(),
            'var': data.var(),
            'range': data.max() - data.min(),
            'iqr': data.quantile(0.75) - data.quantile(0.25),
            'mad': data.mad(), # Mean Absolute Deviation

            # Position
            'min': data.min(),
            'q25': data.quantile(0.25),
            'q50': data.median(),
            'q75': data.quantile(0.75),
            'max': data.max(),

            # Shape
            'skewness': data.skew(),
            'kurtosis': data.kurtosis(),

            # Robust statistics
            'trimmed_mean_5': stats.trim_mean(data, 0.05),

```

```

        'trimmed_mean_10': stats.trim_mean(data, 0.10),
    }

    # Distribution Analysis
    distribution_analysis = self._analyze_distribution(data)
    stats_summary.update(distribution_analysis)

    # Outlier Analysis
    outlier_analysis = self._analyze_outliers(data)
    stats_summary.update(outlier_analysis)

    # Create visualizations
    if create_plots:
        self._plot_numeric_variable(data, column)

    # Interpretation and recommendations
    interpretation = self._interpret_numeric_variable(stats_summary)

    print(f"=== NUMERIC ANALYSIS: {column} ===")
    self._print_numeric_summary(stats_summary, interpretation)

    return {
        'statistics': stats_summary,
        'interpretation': interpretation
    }

def analyze_categorical_variable(self, column, create_plots=True):
    """Comprehensive analysis of categorical variables"""

    if column not in self.df.columns:
        return f"Column '{column}' not found"

    data = self.df[column].dropna()

    if len(data) == 0:
        return f"No non-null values in column '{column}'"

    # Basic Statistics
    value_counts = data.value_counts()
    value_props = data.value_counts(normalize=True)

    stats_summary = {
        'count': len(data),
        'missing': self.df[column].isnull().sum(),
        'missing_pct': (self.df[column].isnull().sum() / len(self.df)) * 100,
        'unique_count': data.nunique(),
        'most_frequent': value_counts.index[0],
        'most_frequent_count': value_counts.iloc[0],
        'most_frequent_pct': value_props.iloc[0] * 100,
        'least_frequent': value_counts.index[-1],
        'least_frequent_count': value_counts.iloc[-1],
        'least_frequent_pct': value_props.iloc[-1] * 100,
    }

    # Diversity measures
    stats_summary['entropy'] = stats.entropy(value_counts)

```

```

stats_summary['gini_coefficient'] = self._calculate_gini_coefficient(value_counts)

# Cardinality assessment
cardinality_assessment = self._assess_cardinality(stats_summary)
stats_summary.update(cardinality_assessment)

# Distribution uniformity
expected_freq = len(data) / data.nunique()
chi2_stat, p_value = stats.chisquare(value_counts)

stats_summary['uniformity_test'] = {
    'chi2_statistic': chi2_stat,
    'p_value': p_value,
    'is_uniform': p_value > 0.05
}

# Create visualizations
if create_plots:
    self._plot_categorical_variable(data, column, value_counts)

# Interpretation
interpretation = self._interpret_categorical_variable(stats_summary)

print(f"=== CATEGORICAL ANALYSIS: {column} ===")
self._print_categorical_summary(stats_summary, interpretation)

return {
    'statistics': stats_summary,
    'value_counts': value_counts,
    'value_proportions': value_props,
    'interpretation': interpretation
}

def analyze_datetime_variable(self, column, create_plots=True):
    """Analysis of datetime variables"""

    if column not in self.df.columns:
        return f"Column '{column}' not found"

    # Convert to datetime if not already
    if not pd.api.types.is_datetime64_any_dtype(self.df[column]):
        try:
            datetime_data = pd.to_datetime(self.df[column])
        except:
            return f"Cannot convert column '{column}' to datetime"
    else:
        datetime_data = self.df[column]

    data = datetime_data.dropna()

    if len(data) == 0:
        return f"No non-null datetime values in column '{column}'"

    # Basic datetime statistics
    stats_summary = {
        'count': len(data),

```

```

        'missing': datetime_data.isnull().sum(),
        'missing_pct': (datetime_data.isnull().sum() / len(self.df)) * 100,
        'min_date': data.min(),
        'max_date': data.max(),
        'date_range': data.max() - data.min(),
        'unique_dates': data.nunique(),
    }

    # Temporal patterns
    stats_summary.update({
        'year_range': data.dt.year.nunique(),
        'month_distribution': data.dt.month.value_counts().to_dict(),
        'day_of_week_distribution': data.dt.dayofweek.value_counts().to_dict(),
        'hour_distribution': data.dt.hour.value_counts().to_dict() if any(data.dt.time
    })

    # Trend analysis (if enough data points)
    if len(data) > 10:
        # Simple trend test using Mann-Kendall
        trend_analysis = self._analyze_temporal_trend(data)
        stats_summary['trend_analysis'] = trend_analysis

    # Create visualizations
    if create_plots:
        self._plot_datetime_variable(data, column)

    # Interpretation
    interpretation = self._interpret_datetime_variable(stats_summary)

    print(f"=== DATETIME ANALYSIS: {column} ===")
    self._print_datetime_summary(stats_summary, interpretation)

    return {
        'statistics': stats_summary,
        'interpretation': interpretation
    }

def _analyze_distribution(self, data):
    """Analyze distribution characteristics"""

    # Normality tests
    normality_tests = {}

    if len(data) >= 8: # Minimum for Shapiro-Wilk
        if len(data) <= 5000: # Shapiro-Wilk limit
            shapiro_stat, shapiro_p = shapiro(data)
            normality_tests['shapiro_wilk'] = {
                'statistic': shapiro_stat,
                'p_value': shapiro_p,
                'is_normal': shapiro_p > 0.05
            }

        # Jarque-Bera test (good for large samples)
        jb_stat, jb_p = jarque_bera(data)
        normality_tests['jarque_bera'] = {
            'statistic': jb_stat,

```

```

        'p_value': jb_p,
        'is_normal': jb_p > 0.05
    }

    # Anderson-Darling test
    ad_result = anderson(data, dist='norm')
    normality_tests['anderson_darling'] = {
        'statistic': ad_result.statistic,
        'critical_values': ad_result.critical_values,
        'significance_levels': ad_result.significance_levels
    }

    return {
        'normality_tests': normality_tests
    }

def _analyze_outliers(self, data):
    """Comprehensive outlier analysis"""

    # IQR Method
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    iqr_outliers = data[(data < lower_bound) | (data > upper_bound)]

    # Z-Score Method
    z_scores = np.abs(stats.zscore(data))
    z_outliers = data[z_scores > 3]

    # Modified Z-Score Method (more robust)
    median = data.median()
    mad = stats.median_abs_deviation(data)
    modified_z_scores = 0.6745 * (data - median) / mad
    modified_z_outliers = data[np.abs(modified_z_scores) > 3.5]

    return {
        'outlier_analysis': {
            'iqr_method': {
                'count': len(iqr_outliers),
                'percentage': (len(iqr_outliers) / len(data)) * 100,
                'lower_bound': lower_bound,
                'upper_bound': upper_bound
            },
            'z_score_method': {
                'count': len(z_outliers),
                'percentage': (len(z_outliers) / len(data)) * 100
            },
            'modified_z_score_method': {
                'count': len(modified_z_outliers),
                'percentage': (len(modified_z_outliers) / len(data)) * 100
            }
        }
    }

```

```

    }

def _calculate_gini_coefficient(self, value_counts):
    """Calculate Gini coefficient for categorical distribution"""
    n = len(value_counts)
    if n <= 1:
        return 0

    sorted_counts = np.sort(value_counts.values)
    cumsum = np.cumsum(sorted_counts)

    return (n + 1 - 2 * np.sum(cumsum) / cumsum[-1]) / n

def _assess_cardinality(self, stats_summary):
    """Assess cardinality level"""

    unique_ratio = stats_summary['unique_count'] / stats_summary['count']

    if unique_ratio > 0.9:
        cardinality_level = "Very High (potentially identifier)"
    elif unique_ratio > 0.5:
        cardinality_level = "High"
    elif unique_ratio > 0.1:
        cardinality_level = "Medium"
    else:
        cardinality_level = "Low"

    return {
        'unique_ratio': unique_ratio,
        'cardinality_level': cardinality_level
    }

def _analyze_temporal_trend(self, data):
    """Analyze temporal trends in datetime data"""

    # Create a simple time series
    time_series = data.value_counts().sort_index()

    if len(time_series) < 3:
        return {'trend': 'Insufficient data for trend analysis'}

    # Simple linear trend
    x = np.arange(len(time_series))
    y = time_series.values

    slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

    if p_value < 0.05:
        if slope > 0:
            trend = "Increasing trend"
        else:
            trend = "Decreasing trend"
    else:
        trend = "No significant trend"

    return {

```

```

        'trend': trend,
        'slope': slope,
        'r_squared': r_value**2,
        'p_value': p_value
    }

def _plot_numeric_variable(self, data, column):
    """Create comprehensive plots for numeric variables"""

    fig, axes = plt.subplots(2, 3, figsize=(18, 12))

    # Histogram
    axes[0,0].hist(data, bins=30, alpha=0.7, edgecolor='black')
    axes[0,0].set_title(f'Histogram: {column}')
    axes[0,0].set_xlabel(column)
    axes[0,0].set_ylabel('Frequency')

    # Box plot
    axes[0,1].boxplot(data)
    axes[0,1].set_title(f'Box Plot: {column}')
    axes[0,1].set_ylabel(column)

    # Q-Q plot
    stats.probplot(data, dist="norm", plot=axes[0,2])
    axes[0,2].set_title(f'Q-Q Plot: {column}')

    # Density plot
    data.plot(kind='density', ax=axes[1,0])
    axes[1,0].set_title(f'Density Plot: {column}')
    axes[1,0].set_xlabel(column)

    # Cumulative distribution
    sorted_data = np.sort(data)
    y = np.arange(1, len(sorted_data) + 1) / len(sorted_data)
    axes[1,1].plot(sorted_data, y)
    axes[1,1].set_title(f'Cumulative Distribution: {column}')
    axes[1,1].set_xlabel(column)
    axes[1,1].set_ylabel('Cumulative Probability')

    # Summary statistics visualization
    stats_text = f"""
    Mean: {data.mean():.3f}
    Median: {data.median():.3f}
    Std: {data.std():.3f}
    Skewness: {data.skew():.3f}
    Kurtosis: {data.kurtosis():.3f}
    """
    axes[1,2].text(0.1, 0.5, stats_text, transform=axes[1,2].transAxes,
                   fontsize=12, verticalalignment='center')
    axes[1,2].set_title(f'Summary Statistics: {column}')
    axes[1,2].axis('off')

    plt.tight_layout()
    plt.show()

def _plot_categorical_variable(self, data, column, value_counts):

```

```

"""Create comprehensive plots for categorical variables"""

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Bar plot
value_counts.head(10).plot(kind='bar', ax=axes[0])
axes[0].set_title(f'Top 10 Categories: {column}')
axes[0].set_xlabel(column)
axes[0].set_ylabel('Count')
axes[0].tick_params(axis='x', rotation=45)

# Pie chart (top 5 categories)
top_5 = value_counts.head(5)
if len(value_counts) > 5:
    other_sum = value_counts.iloc[5:].sum()
    top_5['Other'] = other_sum

axes[1].pie(top_5.values, labels=top_5.index, autopct='%1.1f%%')
axes[1].set_title(f'Distribution: {column}')

# Frequency distribution
proportions = value_counts / len(data)
proportions.head(10).plot(kind='bar', ax=axes[2])
axes[2].set_title(f'Proportions: {column}')
axes[2].set_xlabel(column)
axes[2].set_ylabel('Proportion')
axes[2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

def _plot_datetime_variable(self, data, column):
    """Create plots for datetime variables"""

    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # Timeline
    data.value_counts().sort_index().plot(ax=axes[0,0])
    axes[0,0].set_title(f'Timeline: {column}')
    axes[0,0].set_xlabel('Date')
    axes[0,0].set_ylabel('Count')

    # Monthly distribution
    monthly_counts = data.dt.month.value_counts().sort_index()
    monthly_counts.plot(kind='bar', ax=axes[0,1])
    axes[0,1].set_title(f'Monthly Distribution: {column}')
    axes[0,1].set_xlabel('Month')
    axes[0,1].set_ylabel('Count')

    # Day of week distribution
    dow_counts = data.dt.dayofweek.value_counts().sort_index()
    days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
    dow_counts.index = [days[i] for i in dow_counts.index]
    dow_counts.plot(kind='bar', ax=axes[1,0])
    axes[1,0].set_title(f'Day of Week Distribution: {column}')
    axes[1,0].set_xlabel('Day of Week')

```

```

axes[1,0].set_ylabel('Count')

# Yearly distribution (if multiple years)
if data.dt.year.nunique() > 1:
    yearly_counts = data.dt.year.value_counts().sort_index()
    yearly_counts.plot(kind='bar', ax=axes[1,1])
    axes[1,1].set_title(f'Yearly Distribution: {column}')
    axes[1,1].set_xlabel('Year')
    axes[1,1].set_ylabel('Count')
else:
    axes[1,1].text(0.5, 0.5, 'Single year data',
                  ha='center', va='center', transform=axes[1,1].transAxes)
    axes[1,1].set_title('Yearly Distribution: N/A')

plt.tight_layout()
plt.show()

def _interpret_numeric_variable(self, stats):
    """Generate interpretation for numeric variables"""

    interpretations = []

    # Skewness interpretation
    skewness = stats['skewness']
    if abs(skewness) < 0.5:
        interpretations.append("Distribution is approximately symmetric")
    elif skewness > 0.5:
        interpretations.append("Distribution is right-skewed (positive skew)")
    else:
        interpretations.append("Distribution is left-skewed (negative skew)")

    # Kurtosis interpretation
    kurtosis = stats['kurtosis']
    if kurtosis > 0:
        interpretations.append("Distribution has heavy tails (leptokurtic)")
    elif kurtosis < 0:
        interpretations.append("Distribution has light tails (platykurtic)")

    # Outlier interpretation
    outlier_pct = stats['outlier_analysis']['iqr_method']['percentage']
    if outlier_pct > 5:
        interpretations.append(f"High number of outliers detected ({outlier_pct:.1f}%")
    elif outlier_pct > 1:
        interpretations.append(f"Moderate outliers present ({outlier_pct:.1f}%)")

    # Missing data interpretation
    if stats['missing_pct'] > 10:
        interpretations.append(f"High missing data rate ({stats['missing_pct']:.1f}%")

    return interpretations

def _interpret_categorical_variable(self, stats):
    """Generate interpretation for categorical variables"""

    interpretations = []

```

```

# Cardinality interpretation
interpretations.append(f"Cardinality: {stats['cardinality_level']}")

# Concentration interpretation
if stats['most_frequent_pct'] > 80:
    interpretations.append("Highly concentrated - one category dominates")
elif stats['most_frequent_pct'] > 50:
    interpretations.append("Moderately concentrated")
else:
    interpretations.append("Well distributed across categories")

# Uniformity interpretation
if stats['uniformity_test']['is_uniform']:
    interpretations.append("Distribution is approximately uniform")
else:
    interpretations.append("Distribution is not uniform")

return interpretations

def _interpret_datetime_variable(self, stats):
    """Generate interpretation for datetime variables"""

    interpretations = []

    # Date range interpretation
    date_range = stats['date_range']
    if date_range.days < 30:
        interpretations.append("Short time period (less than 1 month)")
    elif date_range.days < 365:
        interpretations.append("Medium time period (less than 1 year)")
    else:
        interpretations.append(f"Long time period ({date_range.days} days)")

    # Trend interpretation
    if 'trend_analysis' in stats:
        interpretations.append(f"Temporal trend: {stats['trend_analysis']['trend']}")

    return interpretations

def _print_numeric_summary(self, stats, interpretation):
    """Print formatted numeric summary"""

    print(f"Count: {stats['count']}")
    print(f"Missing: {stats['missing']} ({stats['missing_pct']:.1f}%)")
    print(f"Mean: {stats['mean']:.3f}")
    print(f"Median: {stats['median']:.3f}")
    print(f"Std: {stats['std']:.3f}")
    print(f"Skewness: {stats['skewness']:.3f}")
    print(f"Kurtosis: {stats['kurtosis']:.3f}")
    print(f"Range: {stats['range']:.3f}")
    print(f"IQR: {stats['iqr']:.3f}")

    if 'outlier_analysis' in stats:
        outlier_pct = stats['outlier_analysis']['iqr_method']['percentage']
        print(f"Outliers (IQR method): {stats['outlier_analysis']['iqr_method']['cour

```

```

        print("\nInterpretation:")
        for interp in interpretation:
            print(f"    • {interp}")

def _print_categorical_summary(self, stats, interpretation):
    """Print formatted categorical summary"""

    print(f"Count: {stats['count']}")
    print(f"Missing: {stats['missing']} ({stats['missing_pct']:.1f}%)")
    print(f"Unique categories: {stats['unique_count']}")
    print(f"Most frequent: '{stats['most_frequent']}' ({stats['most_frequent_count']}")
    print(f"Least frequent: '{stats['least_frequent']}' ({stats['least_frequent_count']}")
    print(f"Entropy: {stats['entropy']:.3f}")
    print(f"Gini coefficient: {stats['gini_coefficient']:.3f}")

    print("\nInterpretation:")
    for interp in interpretation:
        print(f"    • {interp}")

def _print_datetime_summary(self, stats, interpretation):
    """Print formatted datetime summary"""

    print(f"Count: {stats['count']}")
    print(f"Missing: {stats['missing']} ({stats['missing_pct']:.1f}%)")
    print(f>Date range: {stats['min_date']} to {stats['max_date']}")
    print(f>Time span: {stats['date_range']}")
    print(f"Unique dates: {stats['unique_dates']}")

    print("\nInterpretation:")
    for interp in interpretation:
        print(f"    • {interp}")

# Example usage
def demonstrate_univariate_analysis():
    # Load sample dataset
    tips = sns.load_dataset('tips')

    analyzer = UnivariateAnalysis(tips)

    # Analyze numeric variable
    print("NUMERIC VARIABLE ANALYSIS:")
    numeric_analysis = analyzer.analyze_numeric_variable('total_bill')

    print("\n" + "="*60 + "\n")

    # Analyze categorical variable
    print("CATEGORICAL VARIABLE ANALYSIS:")
    categorical_analysis = analyzer.analyze_categorical_variable('day')

    return analyzer, tips

# Run demonstration
univariate_analyzer, sample_data = demonstrate_univariate_analysis()

```

Quick Reference Guide

EDA Process Checklist

```
eda_checklist = {  
    'data_understanding': [  
        'Check data shape and structure',  
        'Identify data types',  
        'Assess memory usage',  
        'Review column names'  
    ],  
    'data_quality': [  
        'Count missing values',  
        'Identify duplicates',  
        'Check for outliers',  
        'Validate data ranges'  
    ],  
    'univariate_analysis': [  
        'Summary statistics',  
        'Distribution analysis',  
        'Outlier detection',  
        'Normality testing'  
    ],  
    'bivariate_analysis': [  
        'Correlation analysis',  
        'Categorical associations',  
        'Group comparisons',  
        'Relationship strength'  
    ],  
    'visualization': [  
        'Choose appropriate charts',  
        'Create informative titles',  
        'Add proper labels',  
        'Consider color accessibility'  
    ]  
}
```

Common EDA Mistakes to Avoid

1. **Not checking data quality first** - Always assess completeness and accuracy
2. **Ignoring data types** - Ensure proper data types before analysis
3. **Over-relying on correlation** - Remember "correlation $\neq$ causation"
4. **Not considering business context** - Domain knowledge is crucial
5. **Creating too many visualizations** - Focus on meaningful insights
6. **Ignoring outliers** - Investigate rather than automatically remove
7. **Not documenting findings** - Keep track of insights and decisions

Visualization Selection Guide

```
plot_selection = {
    'numeric_single': ['histogram', 'box_plot', 'density_plot'],
    'categorical_single': ['bar_chart', 'pie_chart', 'count_plot'],
    'numeric_vs_numeric': ['scatter_plot', 'line_plot', 'joint_plot'],
    'categorical_vs_numeric': ['box_plot', 'violin_plot', 'strip_plot'],
    'categorical_vs_categorical': ['stacked_bar', 'heatmap', 'mosaic_plot'],
    'time_series': ['line_plot', 'area_plot', 'seasonal_decomposition']
}
```

Statistical Test Selection

```
test_selection = {
    'normality': ['shapiro_wilk', 'jarque_bera', 'anderson_darling'],
    'two_groups_numeric': ['t_test', 'mann_whitney_u', 'welch_t_test'],
    'multiple_groups_numeric': ['anova', 'kruskal_wallis'],
    'categorical_association': ['chi_square', 'fishers_exact'],
    'correlation': ['pearson', 'spearman', 'kendall']
}
```

This comprehensive EDA cheat sheet provides a systematic approach to exploratory data analysis. Use it as a reference for conducting thorough data exploration and generating meaningful insights from your datasets.